

MCP Servers (Model Context Protocol)

Introduced: v1.0.52 (May 2025) | Latest enhancements: v2.1.76

What It Is

MCP (Model Context Protocol) is an open standard that lets Claude Code connect to external tools, databases, and APIs. Instead of Claude only having access to your local filesystem and terminal, MCP servers give it access to GitHub, Sentry, Stripe, PostgreSQL, Slack — anything with an MCP server implementation. You configure servers once, and their tools appear automatically in Claude's toolkit.

Why It Matters

Without MCP, Claude Code is limited to reading files and running commands. With MCP:

- Query production databases directly from conversation
- Create GitHub issues, review PRs, manage repos
- Monitor errors in Sentry, track tasks in Linear
- Build custom integrations for internal tools
- Share tool configurations across your team via `.mcp.json`

Quick Start (60 Seconds)

```
# Add a remote HTTP server (e.g., Sentry)
claude mcp add --transport http sentry https://mcp.sentry.dev/mcp

# Add a local stdio server (e.g., PostgreSQL)
claude mcp add --transport stdio mydb -- npx -y @bytebase/dbhub \
  --dsn "postgresql://user:pass@localhost:5432/mydb"

# List configured servers
claude mcp list

# Inside Claude Code, authenticate and manage
/mcp
```

Transport Types

Type	Use Case	Example
HTTP	Cloud APIs, remote services (recommended)	<code>--transport http stripe https://mcp.stripe.com/mcp</code>
SSE	Legacy remote services (deprecated)	<code>--transport sse asana https://mcp.asana.com/sse</code>
Stdio	Local binaries, custom scripts	<code>--transport stdio db --python server.py</code>

Configuration

CLI Commands

Add servers

```
claude mcp add --transport http <name> <url>
claude mcp add --transport http <name> <url> --header "Authorization: Bearer TOKEN"
claude mcp add --transport stdio <name> -- <command> [args]
claude mcp add --transport stdio <name> --env API_KEY=xxx -- npx -y server
```

Scope: where the config is stored

```
claude mcp add --transport http github --scope user <url>      # ~/.claude.json (all projects)
claude mcp add --transport http github --scope project <url>   # .mcp.json (team-shared)
claude mcp add --transport http github --scope local <url>     # local only (default)
```

Manage

```
claude mcp list          # List all servers
claude mcp get <name>    # Server details
claude mcp remove <name> # Delete server
claude mcp add-from-claude-desktop # Import from Claude Desktop
claude mcp reset-project-choices  # Re-approve project servers
```

Project Config: .mcp.json

Check this into git so your whole team gets the same tools:

```
{
  "mcpServers": {
    "company-api": {
      "type": "http",
      "url": "${API_BASE_URL:-https://api.company.com}/mcp",
      "headers": {
        "Authorization": "Bearer ${API_KEY}"
      }
    },
    "postgres": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@bytebase/dbhub"],
      "env": {
        "PGHOST": "${DB_HOST}",
        "PGPASSWORD": "${DB_PASSWORD}"
      }
    }
  }
}
```

Environment variables use \${VAR} or \${VAR:-default} syntax.

OAuth Authentication

Automatic OAuth (server supports dynamic registration)

```
claude mcp add --transport http sentry https://mcp.sentry.dev/mcp
```

Then inside Claude Code:

/mcp → Authenticate → Browser opens → Done

```
# Pre-configured OAuth (manual registration)
claude mcp add --transport http myserver \
  --client-id abc123 --client-secret \
  --callback-port 8080 https://mcp.example.com/mcp
```

Real-World Workflows

Workflow 1: Query DB + Create GitHub Issue

```
# Setup
claude mcp add --transport stdio pg -- npx -y @bytebase/dbhub \
  --dsn "postgresql://readonly:pass@prod.db.com:5432/analytics"
claude mcp add --transport http github https://api.githubcopilot.com/mcp/

# Use
> "Find customers without purchases in 30 days and create
> a GitHub issue for the sales team with the list"
# Claude queries DB, formats results, creates issue automatically
```

Workflow 2: Monitor Errors and Triage

```
claude mcp add --transport http sentry https://mcp.sentry.dev/mcp
# /mcp → Authenticate Sentry

> "Show top 5 errors from today. For critical ones,
> find the relevant code and suggest fixes."
# Claude fetches errors from Sentry, reads local code, proposes patches
```

Workflow 3: Custom Internal Tool

```
# my_mcp_server.py – expose internal API as MCP tools
from mcp import import Server
app = Server("internal-api")

@app.tool()
def get_feature_flags(environment: str) -> dict:
    """Get feature flags for an environment"""
    return fetch_flags(environment)

claude mcp add --transport stdio internal -- python my_mcp_server.py
```

MCP Elicitation (v2.1.76)

MCP servers can now request structured input from the user mid-task. Instead of Claude guessing at configuration values, the MCP server surfaces an interactive dialog with form fields or a browser URL.

How It Works

User prompt → Claude calls MCP tool → MCP server needs input
→ Elicitation dialog appears → User fills form → Response sent back
→ MCP server continues with user-provided data

Example: Sentry Asks Which Project

You: "Show me today's errors"

Claude calls `mcp__sentry__list_errors`

→ Sentry MCP: "Which project?" [dropdown: frontend, backend, mobile]

→ You select: backend

→ Sentry returns errors for backend project

Hook Integration

Two new hooks intercept elicitation:

Hook	Purpose
Elicitation	Intercept/block MCP input requests before they reach the user
ElicitationResult	Override/validate user responses before sending back to MCP

```
{
  "hooks": {
    "Elicitation": [
      {
        "matcher": "sentry",
        "hooks": [
          {
            "type": "command",
            "command": "echo 'Auto-selecting project: backend'; exit 0"
          }
        ]
      }
    ]
  }
}
```

Permissions & Tool Search

Permission Rules

```
{
  "permissions": {
    "allow": ["MCP(github:*)"],
    "deny": ["MCP(stripe:*)"]
  }
}
```

Tool Search (Context Optimization)

When many MCP servers are configured, tool definitions can consume too much context. Tool search defers loading until needed:

```
ENABLE_TOOL_SEARCH=auto claude      # Auto-activate at 10% threshold
ENABLE_TOOL_SEARCH=auto:5 claude     # Custom 5% threshold
ENABLE_TOOL_SEARCH=true claude       # Always enabled
```

Output Limits

`export MAX_MCP_OUTPUT_TOKENS=50000` *# Default: 25,000*

How It Interacts With Other Features

Feature	Interaction
Plugins	Plugins can bundle MCP servers in <code>.mcp.json</code> — auto-start when enabled
Skills	Skills can use <code>!`command`</code> to fetch MCP data in preprocessing
Hooks	PreToolUse hooks can match <code>mcp__servername__toolname</code> patterns
Agent Teams	Teammates inherit lead's MCP servers
Remote Control	Remote sessions access your local MCP servers (runs locally)
Worktrees	Each worktree inherits MCP config from parent repo

Gotchas & Tips

1. **Windows stdio servers** — wrap with `cmd /c npx` on native Windows (not WSL).
 2. **Scope precedence** — `local > project > user`. A local “github” server overrides a user-scoped “github”.
 3. **Environment variables in `.mcp.json`** — if a `${VAR}` is unset and has no default, config fails to parse.
 4. **Startup timeout** — set `MCP_TIMEOUT=10000` (ms) for slow-starting servers.
 5. **Large output warnings** — increase `MAX_MCP_OUTPUT_TOKENS` if server returns big payloads.
 6. **Project `.mcp.json` requires approval** — first use prompts trust dialog. Reset with `claude mcp reset-project-choices`.
-

Version History

Version	Date	Change
Nov 2024	—	MCP protocol announced by Anthropic
v1.0.52	May 2025	MCP server support added to Claude Code
v2.1.0	Jan 2026	<code>list_changed</code> notifications for dynamic tool updates
v2.1.46	Feb 2026	MCP connector support for <code>claude.ai</code>
v2.1.49	Feb 2026	OAuth step-up auth, discovery caching
v2.1.50	Feb 2026	<code>/mcp</code> reconnect fix, tool search improvements

Version	Date	Change
v2.1.51	Feb 2026	Environment variable interpolation security fixes
v2.1.76	Mar 2026	MCP Elicitation support, Elicitation/ElicitationResult hooks
v2.1.76	Mar 2026	Deferred tools (ToolSearch) retain schemas after compaction